05 / OVERVIEW

Super-agent System Design

Turn architecture ideas into implementable, verifiable agent workflows

This skill supports persistent, multitask, multichannel agent systems. It guides coding assistants through task ownership, event routing, context, durable state, and recovery, reducing manual session switching and progress synchronization.

For: developers designing personal/team agents, adapting the public prototype, or studying Super-agent architecture.

Deliverables: architecture and contracts, implementation-capability comparisons, acceptance scenarios, and an offline teaching lab.

Where it fits among the five skills

Order	Topic	Development problem
01	Configuration	Turn hard-coded behavior into typed, validated settings with defaults.
02	Lifecycle	Manage dependencies, registrations, and external resources; verify disposal and reloads.
03	Services	Separate contracts, Providers, and model-tool entrypoints for replaceable implementations.
04	Packaging	Create installable bundles; inspect archives and profile activation.
05	Super-agent	Compose these capabilities into persistent task coordination.

Three verifiable outcomes

1. Explain which messages continue existing work and which require a new task.
2. Explain recovery after Worker interruption and rejection of stale results.
3. Separate final results, decision requests, and ordinary progress to reduce unnecessary notifications.

This is a coding-assistant skill, not a service that runs persistently after installation. Real platform integration and deployment require subsequent implementation.

SYSTEM MODEL

From Messages to Results

Separate coordination, execution, and durable state

The Router retains the global task index and delivery state. Workers retain task materials and execution detail. Durable state should survive the loss of an execution instance.

Component	Responsibility and flow
Event sources	Chat / GitHub / CI -> deduplication -> Router for new-work assessment.
Subscriptions	Follow-up context -> the corresponding Worker; subscriptions do not expand authorization.
Router	Task index and dispatch -> Workers with isolated task contexts.
Durable state	Tasks / input / execution generations; shared persistence boundary.
Outbox	Results and action items -> the appropriate request channel.
External supervision	Liveness / progress / bounded recovery based on durable state and artifacts.

This is a design sketch, not a full call graph of the public prototype. Ordinary progress stays in internal records; information requiring human action enters the delivery path.

Five objects to distinguish

Object	Meaning
Trigger	A source event, deduplicated with a stable key.
Thread	A discussion context that can relate to multiple tasks.
Ticket	Business work with defined scope and authorization.
Assignment	A task bound to one execution; replaceable without losing the task.
Outbox	Durable pending messages and attempt state.

When modifying the public prototype, extend its Runtime / Engine / Assessor / Executor / Outbox boundaries. Engine transactions record state; external effects run after commit.

CAPABILITY BOUNDARIES

Design and Implementation

Distinguish article ideas, code evidence, and proposed additions

The official article explores architecture. The public repository is a research prototype covering part of that design. Start with three columns: article design / code evidence / additions for this task.

Theme	Verified public behavior	Not established by that evidence
Two layers	Main Session and separate per-Ticket Executor Sessions/work directories.	Several subagents equal a complete persistent coordinator.
Events	Trigger deduplication, subscription fan-out, task scope.	Any message may create work or expand permissions.
Asynchrony	Prototype background commands/result wakeups; article says the Harness async loop is not enabled by default.	Installing the prototype enables every async behavior in the article.
Recall	Bounded windows, summary entrypoints, expansion by ID, pagination.	Unlimited context or fully solved long-term memory.
State/recovery	SQLite persistence, task/assignment separation, restart recovery.	Complete cloud state/compute separation or arbitrary multidevice migration.
Supervision	Dedicated-profile operations checks, health and recovery paths.	Complete per-Worker progress assessment and goal-drift correction.

Make versions reviewable evidence

Technical materials were checked on **2026-11-01**. Prototype version: **0.1.0**; evidence commit: **6337fc8**. Check its requirements together with the actual host, rather than borrowing another master snapshot.

Prototype Node: ^22.19.0 || >=24.0.0
Host range: DSH ^0.2.1-alpha.2
This is a checked snapshot, not a universal installation guarantee.

For new platforms, extend existing Source interfaces. Do not invent configuration keys or callable APIs from an architecture diagram.

HANDS-ON USE

Use It in Your Project

Provide a project path, actual version, and target scenario

Choose architecture delivery, prototype modification, or the teaching lab first, then read the resources relevant to that path.

Example invocation

```
Use $deepseek-harness-05-superagent to design a coordination layer for two tasks receiving chat and CI input. Deliver architecture, event contracts, and recovery acceptance first; do not deploy. Distinguish existing prototype behavior from required additions.
```

Resources and when to read them

Resource	Purpose
SKILL.md	Bilingual entrypoint: identify goals, choose a path, define acceptance.
references/architecture.en.md	Design questions, architecture, failure scenarios.
references/prototype-map.en.md	Prototype comparison, code locations, versions, evidence.
references/lab.en.md	Offline lab and the limits of its conclusions.
assets/superagent-lab/	SQLite coordinator, bilingual demo, ten invariant tests.

Run without a model key

In assets/superagent-lab, using Python 3.9 or later:

```
python -X utf8 -m unittest -v test_coordinator.py
python -X utf8 demo.py
```

The lab creates and cleans up its own temporary local database. start_new=True represents an already authorized simulated decision by the caller. The lab implements no authorization policy, real model, or external message transmission.

A Skill ZIP is a coding-assistant skill directory, not an npm package for dsh plugin add. Running the example and installing a real Harness service are separate steps.

ACCEPTANCE AND SOURCES

Verify Useful Behavior

Check business invariants and usable artifacts

Do more than inspect a model's claim of completion. Start with simulated events/senders, then verify real platforms and sustained operation.

Failure or scenario	Expected observation
Duplicate event	No duplicate work; stable identifiers remain consistent.
Interleaved task messages	Correct ownership and isolated contexts.
Worker expiry/reassignment	Inputs retained; old generation cannot overwrite new results.
Process reopening	Durable facts recover; memory/external effects are not assumed restored.
Delivery succeeds, receipt lost	Platform idempotency or explicit duplicate semantics; no unconditional exactly-once promise.
Progress mixed with decisions	Progress stays internal; results and action items reach the right channel.

What the original delivery verified

All five skills passed format, interface-metadata, and internal-link checks. The offline Super-agent lab passed ten tests, including actual SQLite reopening, event deduplication, generation isolation, and durable Outbox behavior. An independent architecture exercise was also completed.

No real Super-agent service was installed or deployed. Offline success does not verify a real async agent loop, cross-machine migration, platform delivery, or a semantic Watchdog. The dated record and bilingual-update checks are in docs/validation-summary.en.md.

Official evidence

[1] Super-agent design article

[2] Prototype README: positioning and scope

[3] Domain model

[3a] Authorization, subscriptions, and scope

[4] Architecture: runtime, engine, execution, delivery

[5] Deployment and operations

Links are clickable. Skill references contain a more detailed source-to-implementation map. Recheck actual target versions and interfaces before use.